

CS395T Presentation

LLMs for Robot Planning, Control, and Reward Design

Lingyun Xiao

10/14

From LLMs to Grounded Robotics

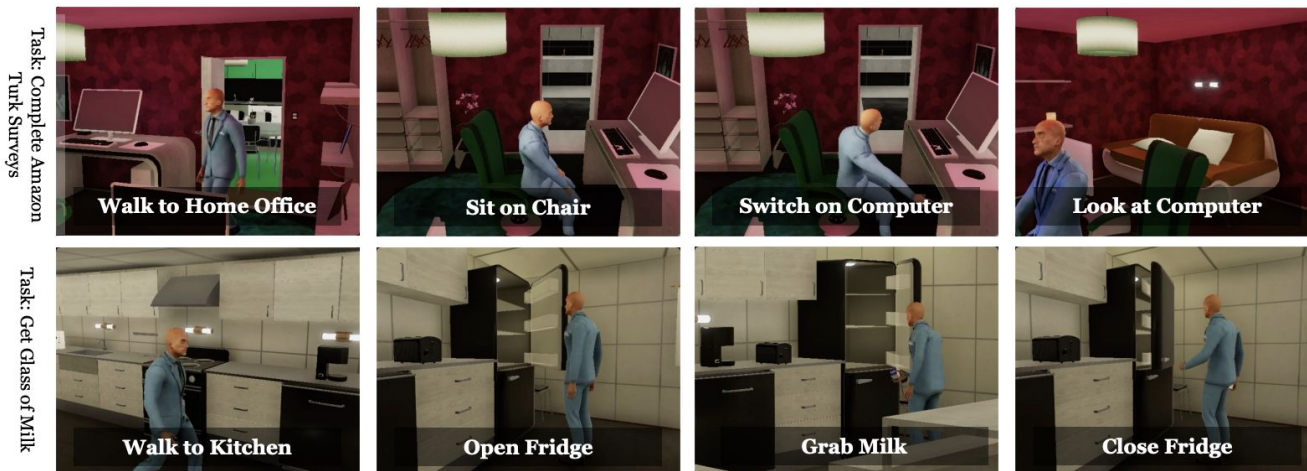
Stage	Key Idea	Representative Paper	Core Question
1. Text as Plans	LLMs produce task decompositions from natural language	<i>LMs as Zero-Shot Planners</i> (Huang et al., 2022)	<i>How far can pretrained knowledge go for open-ended planning?</i>
2. Grounded Feasibility	Combine intent (LLM) and affordance (robot) gives grounded decisions	<i>SayCan</i> (Ahn et al., 2022)	<i>How do we make language-driven plans physically feasible?</i>
3. Code as Policies	LLMs write executable, reactive control programs	<i>Code as Policies</i> (Liang et al., 2022)	<i>Can we compile language directly into robot behavior through code?</i>
4. Rewards as Language	LLMs design and refine reward functions for RL	<i>Eureka</i> (Ma et al., 2023)	<i>Can LLMs automate reward design and improve robot learning?</i>

Language Models as Zero-Shot Planners

- Out-of-box LLMs already encode high-level world **knowledge** on everyday tasks (e.g. go to sleep)
- Real-world applications demand specific **actions**
 - Walk to bedroom → Walk to bed → Sit on bed → Sleep
- Is it possible to extract *plans* from LLMs without training and translate plans into *admissible* actions?

Evaluation Environment: VirtualHome

- Each activity consists of a sequence of action steps
- Each action step is: [action] <arg> (index)
- 42 atomic actions



Evaluation Metrics

- Executability:

Can an action plan be *correctly parsed* and satisfies the *common-sense constraints* of the environment?

- Correctness:

Human judgments on *semantic alignment* with intended task

Use Longest Common Subsequence (LCS) between generated and human-labeled ground truth

Example task: Make a Coffee

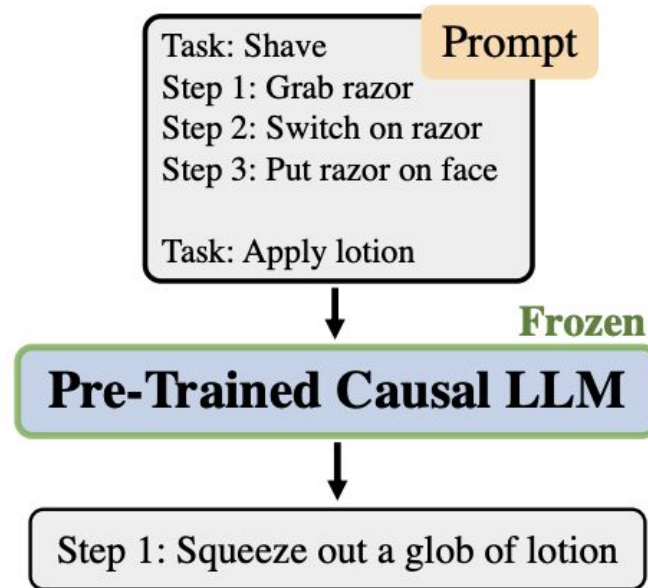
	Executable 	Non-Executable 
Semantically Correct 	pick up the mug → fill with coffee → place on table	press espresso button
Semantically Incorrect 	pick up plate → put in sink → close fridge	dance happily → praise coffee → teleport mug

Querying LLMs for Action Plans

Planning LM

- Use out-of-the-box LLM (e.g. GPT 3)
- Give one correct demonstration
- Sample multiple actions and choose the one with highest mean log probability:

$$\operatorname{argmax}_{X_i} \left(P_{\theta}(X_i) := \frac{1}{n_i} \sum_{j=1}^{n_i} \log p_{\theta}(x_{i,j} | x_{i,<j}) \right)$$



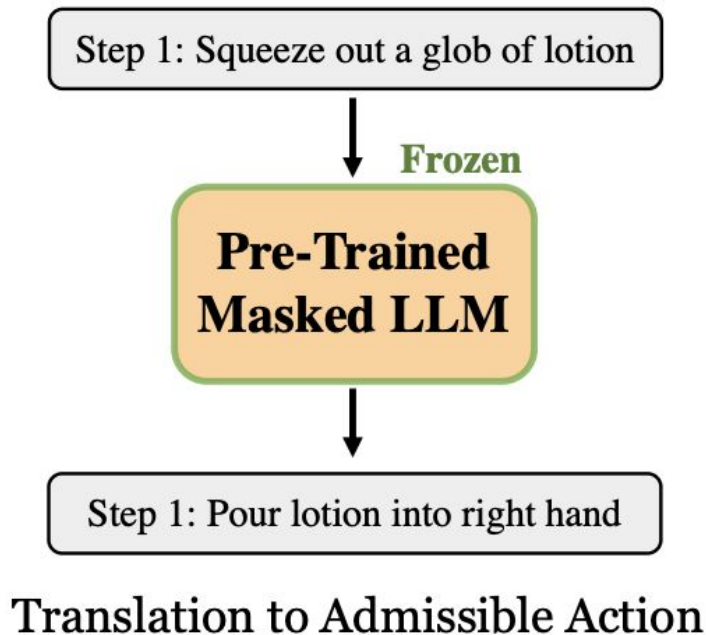
Zero-Shot Planning via Causal LLM

Admissible Action Parsing by Semantic Translation

Translated LM

- Issue: The generated plan might not be directly executable or ambiguous
- BERT-style LM to embed actions
- Calculate cosine-similarity between the generated action and all admissible actions, find the closest

$$C(f(\hat{a}), f(a_e)) := \frac{f(\hat{a}) \cdot f(a_e)}{\|f(\hat{a})\| \|f(a_e)\|}$$



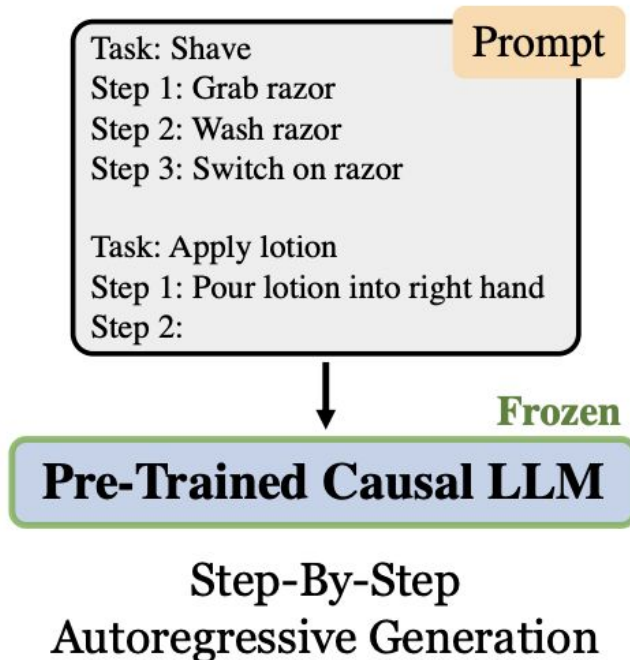
Autoregressive Trajectory Correction

Append translated actions back into the prompt for next step

Maximize the following metric

$$\operatorname{argmax}_{a_e} \left[\max_{\hat{a}} C(f(\hat{a}), f(a_e)) + \beta \cdot P_{\theta}(\hat{a}) \right]$$

All future steps are conditioned on only admissible actions



Dynamic Example Selection

- LLMs need some domain knowledge to generate executable plans, which is difficult by zero-shot planning
- Select the most similar task T and its example plan E from the demonstration set to be used as the example in the prompt
- Helps bias the LLM more appropriately using weak supervision WITHOUT fine-tuning

Results

Language Model	Executability	LCS	Correctness
Vanilla GPT-2 117M	18.66%	3.19%	15.81% (4.90%)
Vanilla GPT-2 1.5B	39.40%	7.78%	29.25% (5.28%)
Vanilla Codex 2.5B	17.62%	15.57%	63.08% (7.12%)
Vanilla GPT-Neo 2.7B	29.92%	11.52%	65.29% (9.08%)
Vanilla Codex 12B	18.07%	16.97%	64.87% (5.41%)
Vanilla GPT-3 13B	25.87%	13.40%	49.44% (8.14%)
Vanilla GPT-3 175B	7.79%	17.82%	77.86% (6.42%)
Human	100.00%	N/A	70.05% (5.44%)
Fine-tuned GPT-3 13B	66.07%	34.08%	64.92% (5.96%)
OUR FINAL METHODS			
Translated Codex 12B	78.57%	24.72%	54.88% (5.90%)
Translated GPT-3 175B	73.05%	24.09%	66.13% (8.38%)

Ablations

Methods	Executability	LCS
Translated Codex 12B	78.57%	24.72%
- w/o Action Translation	31.49%	22.53%
- w/o Dynamic Example	50.86%	22.84%
- w/o Trajectory Correction	55.19%	24.43%
Translated GPT-3 175B	73.05%	24.09%
- w/o Action Translation	36.04%	24.31%
- w/o Dynamic Example	60.82%	22.92%
- w/o Trajectory Correction	40.10%	24.98%

Main Contributions and Limitations

- Without any training, LLMs can be prompted to generate plausible goal-driven action plans, but such plans are frequently **not executable**.
- Propose several tools to **improve executability** of the generated plans without fine-tuning the model.
- However, there is *no real-world grounding*, and often leads to *hallucinations or misalignment* with human intent!
- This is critical in real-world applications such as Robotics.

Do As I Can, Not As I Say: Grounding Language in Robotic Affordances

- LLMs can propose actions but may suggest infeasible ones
- Robots have dynamics, constraints, affordances
- Need to ground language-based plans in what the robot can do, not only what the robot *should* do

Do As I Can, Not As I Say: Grounding Language in Robotic Affordances

Algorithm 1 SayCan

Given: A high level instruction i , state s_0 , and a set of skills Π and their language descriptions ℓ_Π

```

1:  $n = 0, \pi = \emptyset$ 
2: while  $\ell_{\pi_{n-1}} \neq \text{"done"}$  do
3:    $\mathcal{C} = \emptyset$ 
4:   for  $\pi \in \Pi$  and  $\ell_\pi \in \ell_\Pi$  do
5:      $p_\pi^{\text{LLM}} = p(\ell_\pi | i, \ell_{\pi_{n-1}}, \dots, \ell_{\pi_0})$ 
6:      $p_\pi^{\text{affordance}} = p(c_\pi | s_n, \ell_\pi)$ 
7:      $p_\pi^{\text{combined}} = p_\pi^{\text{affordance}} p_\pi^{\text{LLM}}$ 
8:      $\mathcal{C} = \mathcal{C} \cup p_\pi^{\text{combined}}$ 
9:   end for
10:   $\pi_n = \arg \max_{\pi \in \Pi} \mathcal{C}$ 
11:  Execute  $\pi_n(s_n)$  in the environment, updating state  $s_{n+1}$ 
12:   $n = n + 1$ 
13: end while
    
```

The **"Should"** Score

How useful is this skill to the command?

▷ Evaluate scoring of LLM

▷ Evaluate affordance function

The **"Can"** Score

How likely is this skill successfully executed?

Do As I Can, Not As I Say: Grounding Language in Robotic Affordances

Algorithm 1 SayCan

Given: A high level instruction i , state s_0 , and a set of skills Π and their language descriptions ℓ_Π

```

1:  $n = 0, \pi = \emptyset$ 
2: while  $\ell_{\pi_{n-1}} \neq \text{"done"}$  do
3:    $\mathcal{C} = \emptyset$ 
4:   for  $\pi \in \Pi$  and  $\ell_\pi \in \ell_\Pi$  do
5:      $p_\pi^{\text{LLM}} = p(\ell_\pi | i, \ell_{\pi_{n-1}}, \dots, \ell_{\pi_0})$ 
6:      $p_\pi^{\text{affordance}} = p(c_\pi | s_n, \ell_\pi)$ 
7:      $p_\pi^{\text{combined}} = p_\pi^{\text{affordance}} p_\pi^{\text{LLM}}$ 
8:      $\mathcal{C} = \mathcal{C} \cup p_\pi^{\text{combined}}$ 
9:   end for
10:   $\pi_n = \arg \max_{\pi \in \Pi} \mathcal{C}$ 
11:  Execute  $\pi_n(s_n)$  in the environment, updating state  $s_{n+1}$ 
12:   $n = n + 1$ 
13: end while
    
```

The “**Should**” Score

How useful is this skill to the command?

▷ Evaluate scoring of LLM

▷ Evaluate affordance function

Conditional Probability!

How likely does a given skill successfully make progress toward completing the instruction?

The “**Can**” Score

How likely is this skill successfully executed?

Do As I Can, Not As I Say: Grounding Language in Robotic Affordances

Algorithm 1 SayCan

Given: A high level instruction i , state s_0 , and a set of skills Π and their language descriptions ℓ_Π

```

1:  $n = 0, \pi = \emptyset$ 
2: while  $\ell_{\pi_{n-1}} \neq \text{"done"}$  do
3:    $\mathcal{C} = \emptyset$ 
4:   for  $\pi \in \Pi$  and  $\ell_\pi \in \ell_\Pi$  do
5:      $p_\pi^{\text{LLM}} = p(\ell_\pi | i, \ell_{\pi_{n-1}}, \dots, \ell_{\pi_0})$ 
6:      $p_\pi^{\text{affordance}} = p(c_\pi | s_n, \ell_\pi)$ 
7:      $p_\pi^{\text{combined}} = p_\pi^{\text{affordance}} p_\pi^{\text{LLM}}$ 
8:      $\mathcal{C} = \mathcal{C} \cup p_\pi^{\text{combined}}$ 
9:   end for
10:   $\pi_n = \arg \max_{\pi \in \Pi} \mathcal{C}$ 
11:  Execute  $\pi_n(s_n)$  in the environment, updating state  $s_{n+1}$ 
12:   $n = n + 1$ 
13: end while
  
```

The “**Should**” Score

How useful is this skill to the command?

▷ Evaluate scoring of LLM

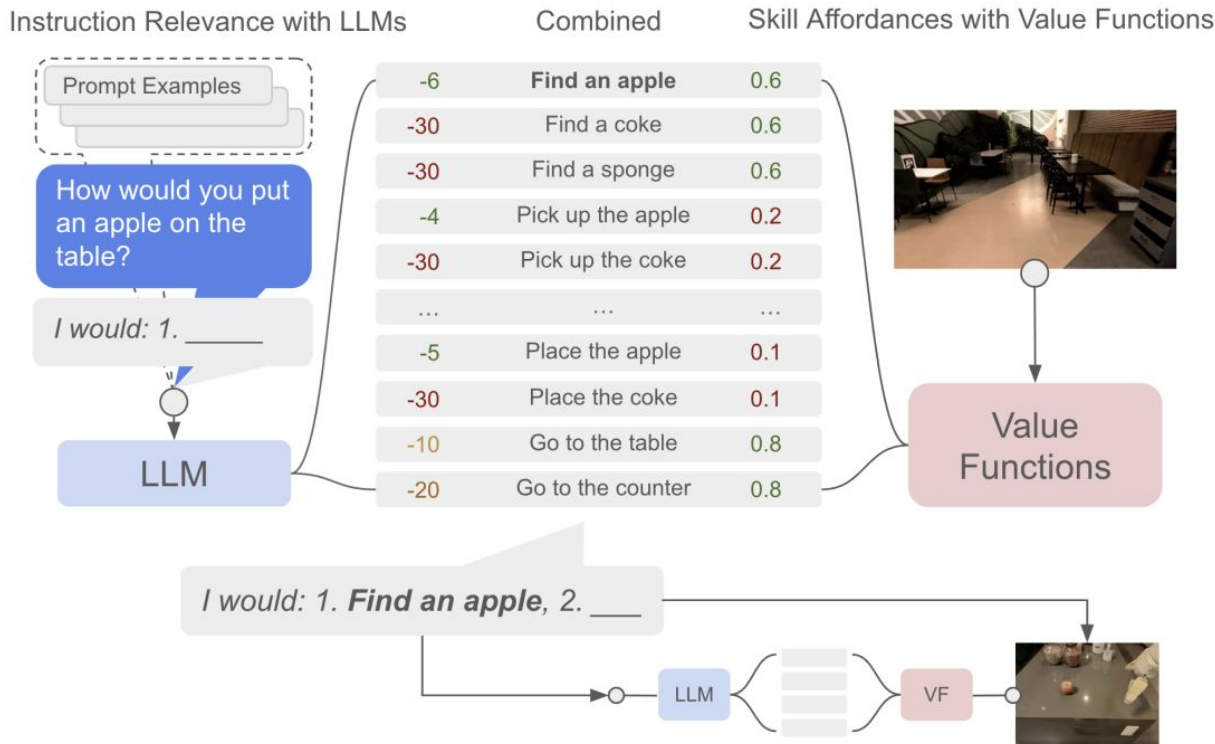
▷ Evaluate affordance function

Conditional Probability!

The “**Can**” Score /
Value Function

How likely is this
skill successfully
executed?

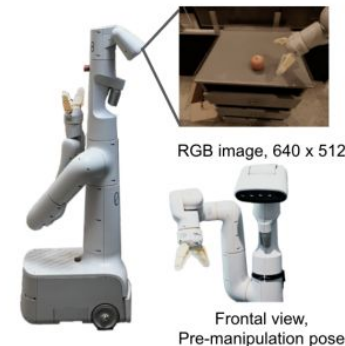
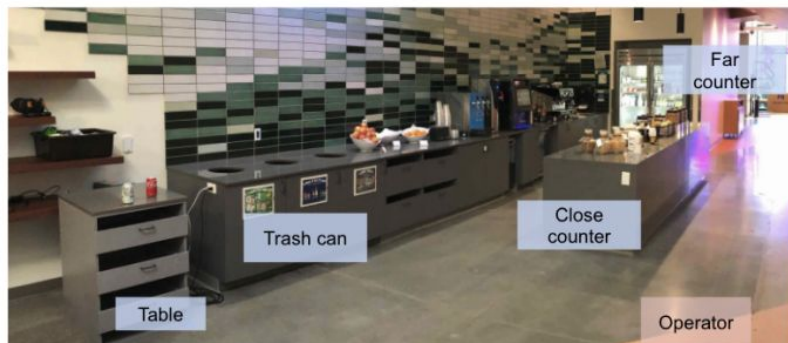
Do As I Can, Not As I Say: Grounding Language in Robotic Affordances



Language-Conditioned Control Policies and Training Low-level Skills

- Predefined **skill** library: e.g. pick, place, move_to
- Learn policies from BC (Behavior Cloning) and RL
- Each skill has a learned **value function**/affordance function
This is the “**Can**” score

Experiment Setup



- Office kitchen environments
- 15 objects and 5 locations
- Mobile manipulator robot
- The LLM model is 540B PaLM

Results

Family	Num	Mock Kitchen		Kitchen		No Affordance		No LLM	
		PaLM-SayCan	PaLM-SayCan	PaLM-SayCan	PaLM-SayCan	No VF	Gen.	BC NL	BC USE
		Plan	Execute	Plan	Execute	Plan	Plan	Execute	Execute
NL Single	15	100%	100%	93%	87%	73%	87%	0%	60%
NL Nouns	15	67%	47%	60%	40%	53%	53%	0%	0%
NL Verbs	15	100%	93%	93%	73%	87%	93%	0%	0%
Structured	15	93%	87%	93%	47%	93%	100%	0%	0%
Embodiment	11	64%	55%	64%	55%	18%	36%	0%	0%
Crowd Sourced	15	87%	87%	73%	60%	67%	80%	0%	0%
Long-Horizon	15	73%	47%	73%	47%	67%	60%	0%	0%
Total	101	84%	74%	81%	60%	67%	74%	0%	9%

Main Contributions and Limitations

- Combining LLM semantic reasoning (“**should**”) with robot affordance/value function(“**can**”) enables grounded, executable robotic actions from natural language.
- However, SayCan only tells the robot what to do because there is a *predefined* skill library.
- The next step is for the LLM to *write* what the robot can do!

Code as Policies: Language Model Programs for Embodied Control

- Predefined robot skill libraries limit flexibility.
- LLMs pretrained on massive code (e.g., Codex, PaLM-Coder) can produce **structured, executable logic**.
- Is it possible to leverage the code-writing abilities of LLMs to translate language into codes as policies, and further enhance **human-robot interaction**?

Prompting Language Model Programs (LMP)

- Hints: import statements that inform the LLM which APIs are available and type hints on how to use those APIs.
- Examples: (Instruction, Code) pairs as few-shot demonstrations

```
# Python script
# get the variable a.
ret_val = a
# find the sum of variables a and b.
ret_val = a + b
# see if any number is divisible by 3 in a list called xs.
ret_val = any(x % 3 == 0 for x in xs)
```

What programs can LMP generate?

- Third/First Party libraries
- Control flows
- LMP composition via nested function calls
- Hierarchical function generation
- Robot policies...

```
objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# while there are blocks with area bigger than 0.2 that are left of the
# red bowl, move them toward the right.
block_names = parse_obj('blocks with area bigger than 0.2 that are
                        left of the red bowl')
while len(block_names) > 0:
    for block_name in block_names:
        target_pos = get_pos(block_name) + np.array([0.1, 0])
        put_first_on_second(block_name, target_pos)
    block_names = parse_obj('blocks with area bigger than 0.2 that are
                            left of the red bowl')
```

Results on Hierarchical code-gen

TABLE I: Hierarchical code-generation solves more problems in **RoboCodeGen** (in % pass rates) and improves with scaling laws (as # model parameters increases).

Method	GPT-3 [12]		Codex [1]	
	6.7B	175B	cushman	davinci
Flat	3	68	54	81
Hierarchical	5	84	57	95

The authors introduce a new benchmark that is robotics-themed

TABLE II: Hierarchical code-gen performs better (% pass rate) on generic coding problems from HumanEval [1]. Greedy is decoding LLM with temperature=0. P@N evaluates correctness across N samples with temperature=0.8.

	Greedy	P@1	P@10	P@100
Flat	45.7	34.9	75.1	90.9
Hierarchical	53.0	39.8	80.6	95.7

Metric is the percentage of the generated code that passes human-written unit tests.

Results on simulated manipulation (UR5e Arm)

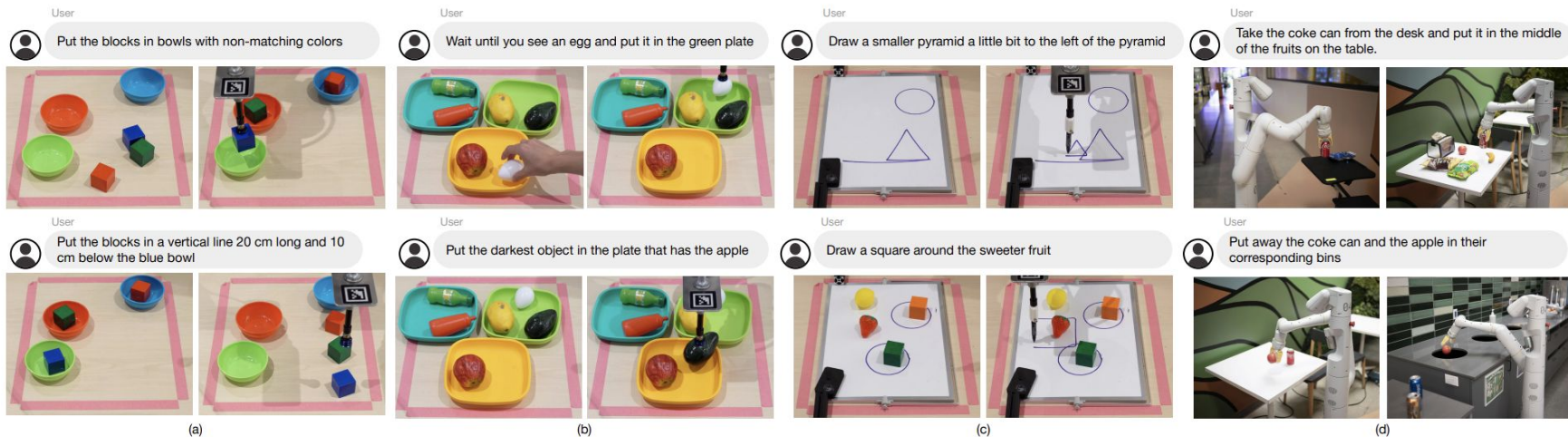
TABLE III: Success rates over task families with 50 trials per task.

Train/Test	Task Family	CLIPort [36]	NL Planner	CaP (ours)
SA SI	Long-Horizon	78.80	86.40	97.20
SA SI	Spatial-Geometric	97.33	N/A	89.30
UA SI	Long-Horizon	36.80	88.00	97.60
UA SI	Spatial-Geometric	0.00	N/A	73.33
UA UI	Long-Horizon	0.00	64.00	80.00
UA UI	Spatial-Geometric	0.01	N/A	62.00

S: Seen
 U: Unseen
 A: Attributes
 I: Instructions

Results on real robot demonstrations

Domain	Task	Capability Shown
UR5e: Draw Shapes	Generate 2D waypoints to draw user-specified shapes	Spatial & geometric reasoning
UR5e: Pick & Place	Manipulate toys via MDETR perception APIs	Object recognition & contextual commands
Mobile Robot: Kitchen	Navigation + manipulation	Long-horizon reactive control across APIs



Main Contributions and Limitations

- Propose a framework where LLMs directly generate executable, reactive code representing robot policies.
 - Introduce hierarchical code generation
 - Provide cross-domain demonstrations
-
- However, the LLM only writes control code, but not *reward* functions in RL, which shape how the robot learns!
-
- The next step is for the LLM to help the robot learn *why* one behavior is better than the other.

Eureka: Human-level Reward Design via Coding LLMs

- Handcrafting reward functions is time-consuming, brittle, and domain-specific.
- Poor reward design leads to reward hacking, misalignment, or failure to learn.
- Can code-oriented LLMs generate structured, verifiable reward functions in RL that replace manual reward design while achieving human-level capabilities in a variety of tasks?

What is Reward Design?

Definition 2.1. (Reward Design Problem ([Singh et al., 2010](#))) A *reward design problem* (RDP) is a tuple $P = \langle M, \mathcal{R}, \pi_M, F \rangle$, where $M = (S, A, T)$ is the *world model* with state space S , action space A , and transition function T . $\mathcal{R}$ is the space of reward functions; $\mathcal{A}_M(\cdot) : \mathcal{R} \rightarrow \Pi$ is a learning algorithm that outputs a policy $\pi : S \rightarrow \Delta(A)$ that optimizes reward $R \in \mathcal{R}$ in the resulting *Markov Decision Process* (MDP), (M, R) ; $F : \Pi \rightarrow \mathbb{R}$ is the *fitness* function that produces a scalar evaluation of any policy, which may only be accessed via policy queries (i.e., evaluate the policy using the ground truth reward function). In an RDP, the goal is to output a reward function $R \in \mathcal{R}$ such that the policy $\pi := \mathcal{A}_M(R)$ that optimizes R achieves the highest fitness score $F(\pi)$.

Reward Generation Problem. In our problem setting, every component within a RDP is specified via code. Then, given a string l that specifies the task, the objective of the reward generation problem is to output a reward function code R such that $F(\mathcal{A}_M(R))$ is maximized.

Eureka Algorithm

Algorithm 1 EUREKA

- 1: **Require:** Task description l , environment code M ,
coding LLM LLM , fitness function F , initial prompt prompt
 - 2: **Hyperparameters:** search iteration N , iteration batch size K
 - 3: **for** N iterations **do**
 - 4: // Sample K reward code from LLM
 - 5: $R_1, \dots, R_k \sim \text{LLM}(l, M, \text{prompt})$
 - 6: // Evaluate reward candidates
 - 7: $s_1 = F(R_1), \dots, s_K = F(R_K)$
 - 8: // Reward reflection
 - 9: $\text{prompt} := \text{prompt} : \text{Reflection}(R_{best}^n, s_{best}^n),$
 where $best = \arg \max_k s_1, \dots, s_K$
 - 10: // Update Eureka reward
 - 11: $R_{\text{Eureka}}, s_{\text{Eureka}} = (R_{best}^n, s_{best}^n), \quad \text{if } s_{best}^n > s_{\text{Eureka}}$
 - 12: **Output:** R_{Eureka}
-

Environment as Context

Algorithm 1 EUREKA

```

1: Require: Task description  $l$ , environment code  $M$ ,
   coding LLM  $\text{LLM}$ , fitness function  $F$ , initial prompt  $\text{prompt}$ 
2: Hyperparameters: search iteration  $N$ , iteration batch size  $K$ 
3: for  $N$  iterations do
4:   // Sample  $K$  reward code from LLM
5:    $R_1, \dots, R_k \sim \text{LLM}(l, M, \text{prompt})$ 
6:   // Evaluate reward candidates
7:    $s_1 = F(R_1), \dots, s_K = F(R_K)$ 
8:   // Reward reflection
9:    $\text{prompt} := \text{prompt} : \text{Reflection}(R_{best}^n, s_{best}^n)$ ,
     where  $best = \arg \max_k s_1, \dots, s_K$ 
10:  // Update Eureka reward
11:   $R_{\text{Eureka}}, s_{\text{Eureka}} = (R_{best}^n, s_{best}^n)$ , if  $s_{best}^n > s_{\text{Eureka}}$ 
12: Output:  $R_{\text{Eureka}}$ 
  
```

Directly ask
LLM to
generate
Python
executable
reward code

Provide
environment
source code
(state + action)

There is NO
task or
env-specific
templates!

Evolutionary Search

Algorithm 1 EUREKA

```

1: Require: Task description  $l$ , environment code  $M$ ,
   coding LLM  $\text{LLM}$ , fitness function  $F$ , initial prompt  $\text{prompt}$ 
2: Hyperparameters: search iteration  $N$ , iteration batch size  $K$ 
3: for  $N$  iterations do
4:   // Sample  $K$  reward code from LLM
5:    $R_1, \dots, R_k \sim \text{LLM}(l, M, \text{prompt})$ 
6:   // Evaluate reward candidates
7:    $s_1 = F(R_1), \dots, s_K = F(R_K)$ 
8:   // Reward reflection
9:    $\text{prompt} := \text{prompt} : \text{Reflection}(R_{best}^n, s_{best}^n)$ ,
     where  $best = \arg \max_k s_1, \dots, s_K$ 
10:  // Update Eureka reward
11:   $R_{\text{Eureka}}, s_{\text{Eureka}} = (R_{best}^n, s_{best}^n)$ , if  $s_{best}^n > s_{\text{Eureka}}$ 
12: Output:  $R_{\text{Eureka}}$ 
    
```

Choose $K = 16$
 contains at
 least one
 executable
 code in first
 generation

Reward Mutation in Iterative Process

```
def compute_reward(object_rot, goal_rot, object_angvel, object_pos, fingertip_pos):
    # Rotation reward
    rot_diff = torch.abs(torch.sum(object_rot * goal_rot, dim=1) - 1) / 2
    - rotation_reward_temp = 20.0
    + rotation_reward_temp = 30.0 Changing hyperparameter
    rotation_reward = torch.exp(-rotation_reward_temp * rot_diff)

    # Distance reward
    + min_distance_temp = 10.0
    min_distance = torch.min(torch.norm(fingertip_pos - object_pos[:, None], dim=2), dim=1).values
    - distance_reward = min_distance
    + uncapped_distance_reward = torch.exp(-min_distance_temp * min_distance)
    + distance_reward = torch.clamp(uncapped_distance_reward, 0.0, 1.0) Changing functional form

    - total_reward = rotation_reward + distance_reward
    + # Angular velocity penalty Adding new component
    + angvel_norm = torch.norm(object_angvel, dim=1)
    + angvel_threshold = 0.5
    + angvel_penalty_temp = 5.0
    + angular_velocity_penalty = torch.where(angvel_norm > angvel_threshold,
    +     torch.exp(-angvel_penalty_temp * (angvel_norm - angvel_threshold)), torch.zeros_like(angvel_norm))
    +
    + total_reward = 0.5 * rotation_reward + 0.3 * distance_reward - 0.2 * angular_velocity_penalty

    reward_components = {
        "rotation_reward": rotation_reward,
        "distance_reward": distance_reward,
    + "angular_velocity_penalty": angular_velocity_penalty,
    }

    return total_reward, reward_components
```

Take the best reward function, reward reflection and the mutation prompt

Generate **K more** i.i.d. Reward outputs

Perform **random restarts** to optimize reward design through different initial samples

Experiments

- LLM: GPT-4 (0314) as backbone.
- Robots & Tasks:
 - 10 robots × 29 tasks in IsaacGym.
 - 9 standard Isaac tasks (quadrupeds, bipeds, quadrotor, arms, dexterous hands).
 - 20 Bi-Dexterity bimanual manipulation tasks.
- Training: Common PPO implementations for all tasks.
- Metrics:
 - Success rate
 - Human-normalized score $\frac{\text{Method} - \text{Sparse}}{|\text{Human} - \text{Sparse}|}$

Eureka beats Human and L2R across all tasks

Baseline

Description

Human

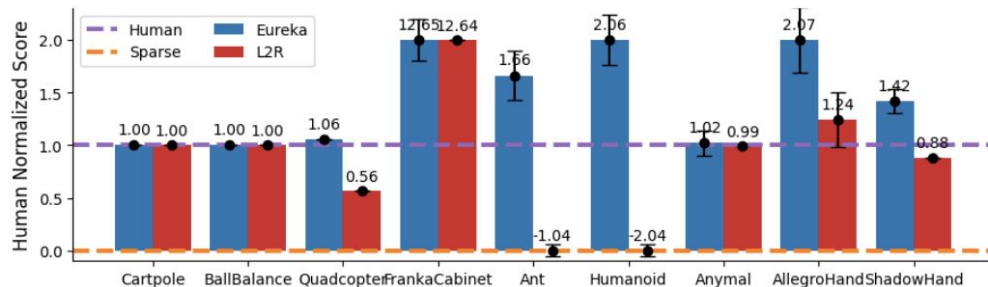
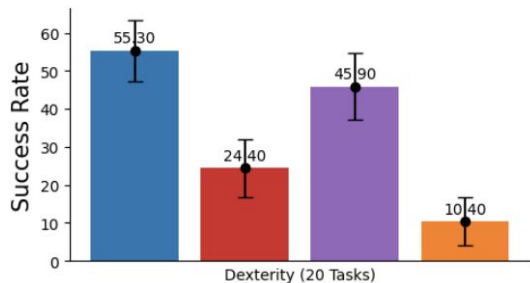
Expert-engineered reward functions from benchmark authors

Sparse

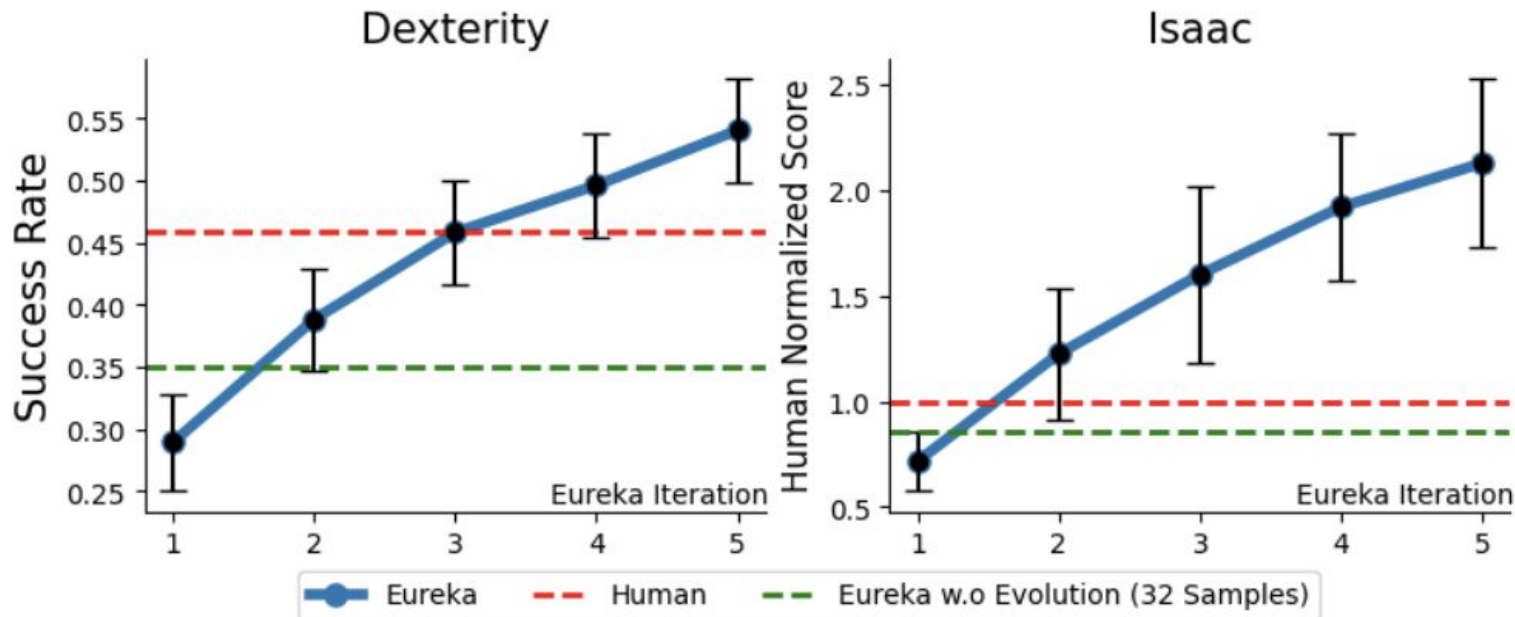
Ground-truth binary/task metrics used for evaluation

L2R (Yu et al., 2023)

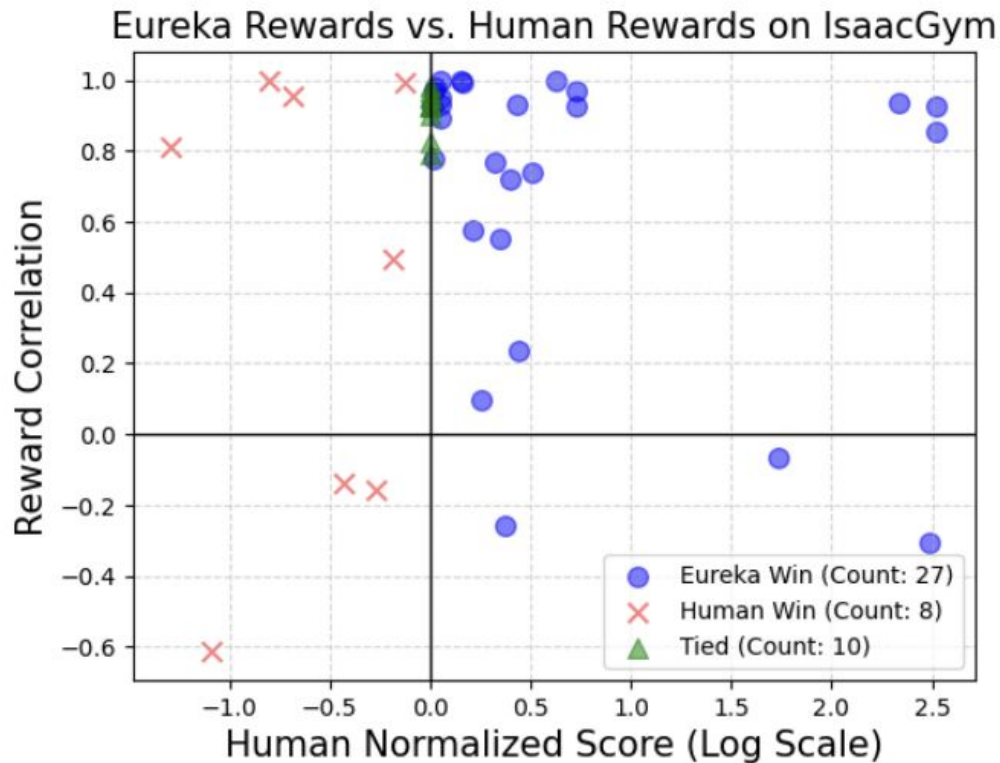
Two-stage template-based LLM reward generation



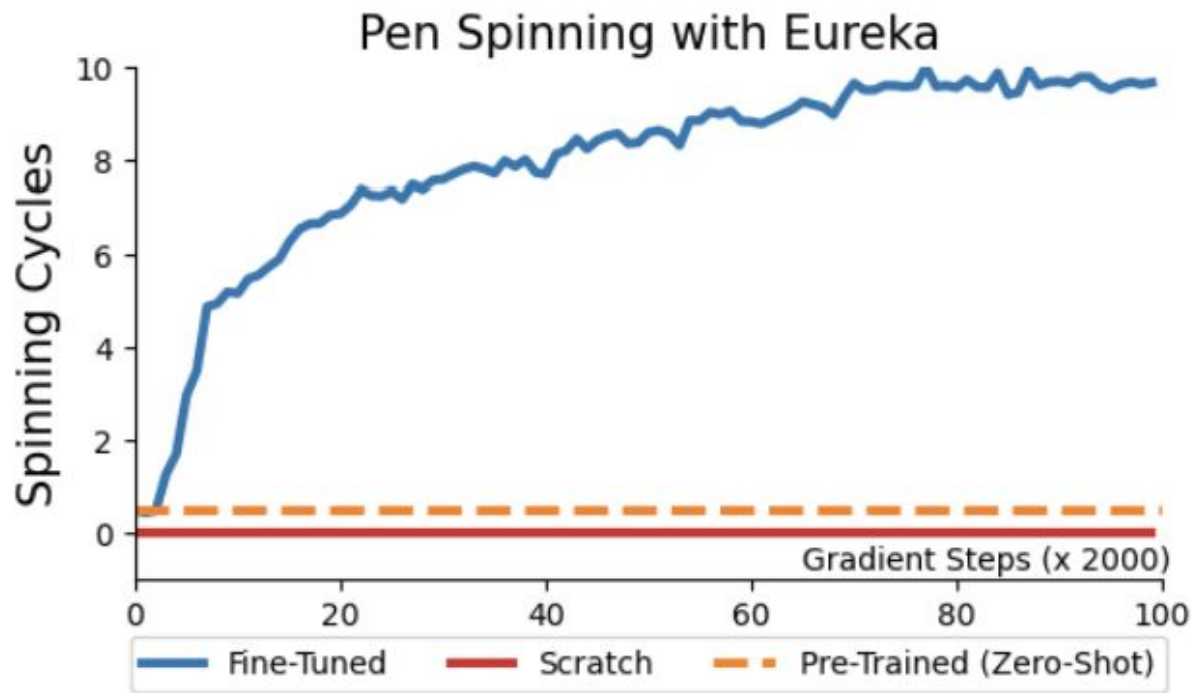
Consistent Reward Improvement



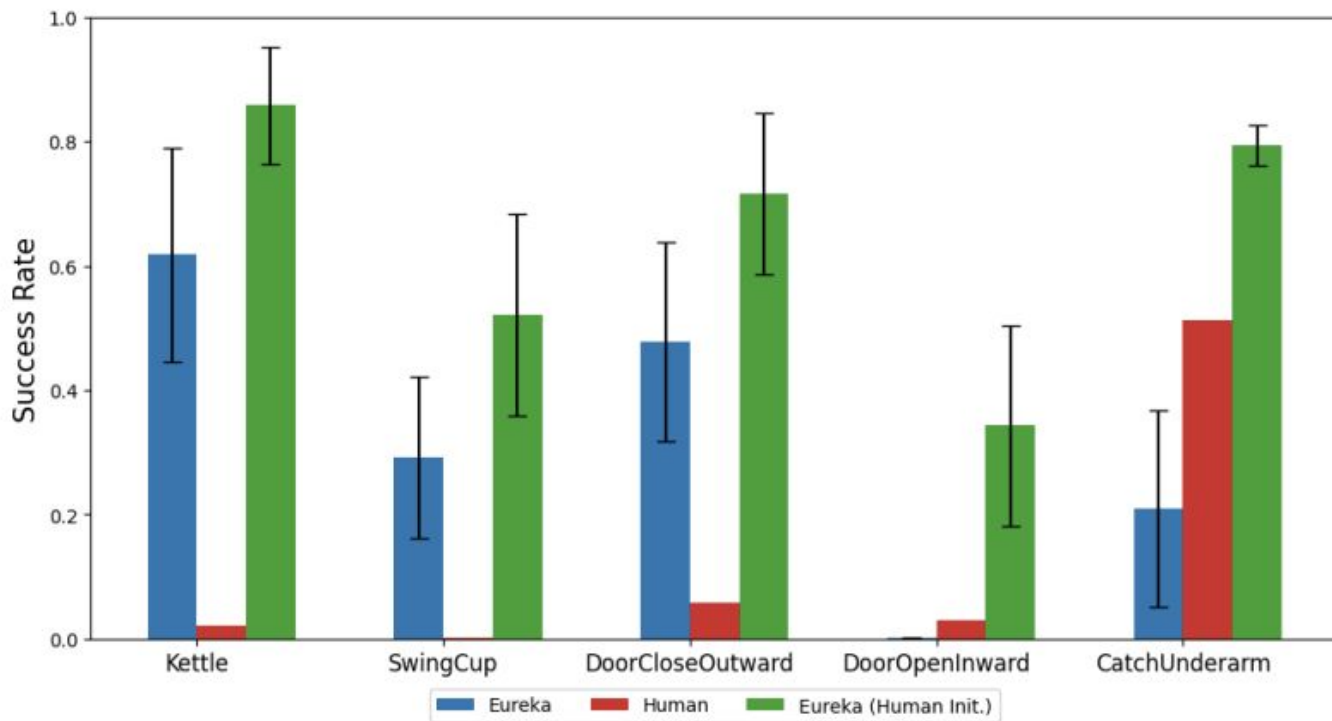
Discovering New Reward Designs



Curriculum Learning on a Challenging Task



Improve on Human Reward Initialization



Induce Human-Aligned Behavior



Method	Forward Velocity	Human Preference
EUREKA	7.53	5/20
EUREKA-HF	5.58	15/20

Main Contributions and Limitations

- Zero-shot reward generation that doesn't require any task-specific templates or engineering
- Outperforms human-designed reward functions and excels in complex environments
- Bridges code generation and reward design for robot learning

Revisit the Roadmap

Stage	Key Idea	Representative Paper	Core Question
1. Text as Plans	LLMs produce task decompositions from natural language	<i>LMs as Zero-Shot Planners</i> (Huang et al., 2022)	<i>How far can pretrained knowledge go for open-ended planning?</i>
2. Grounded Feasibility	Combine intent (LLM) and affordance (robot) gives grounded decisions	<i>SayCan</i> (Ahn et al., 2022)	<i>How do we make language-driven plans physically feasible?</i>
3. Code as Policies	LLMs write executable, reactive control programs	<i>Code as Policies</i> (Liang et al., 2022)	<i>Can we compile language directly into robot behavior through code?</i>
4. Rewards as Language	LLMs design and refine reward functions for RL	<i>Eureka</i> (Ma et al., 2023)	<i>Can LLMs automate reward design and improve robot learning?</i>